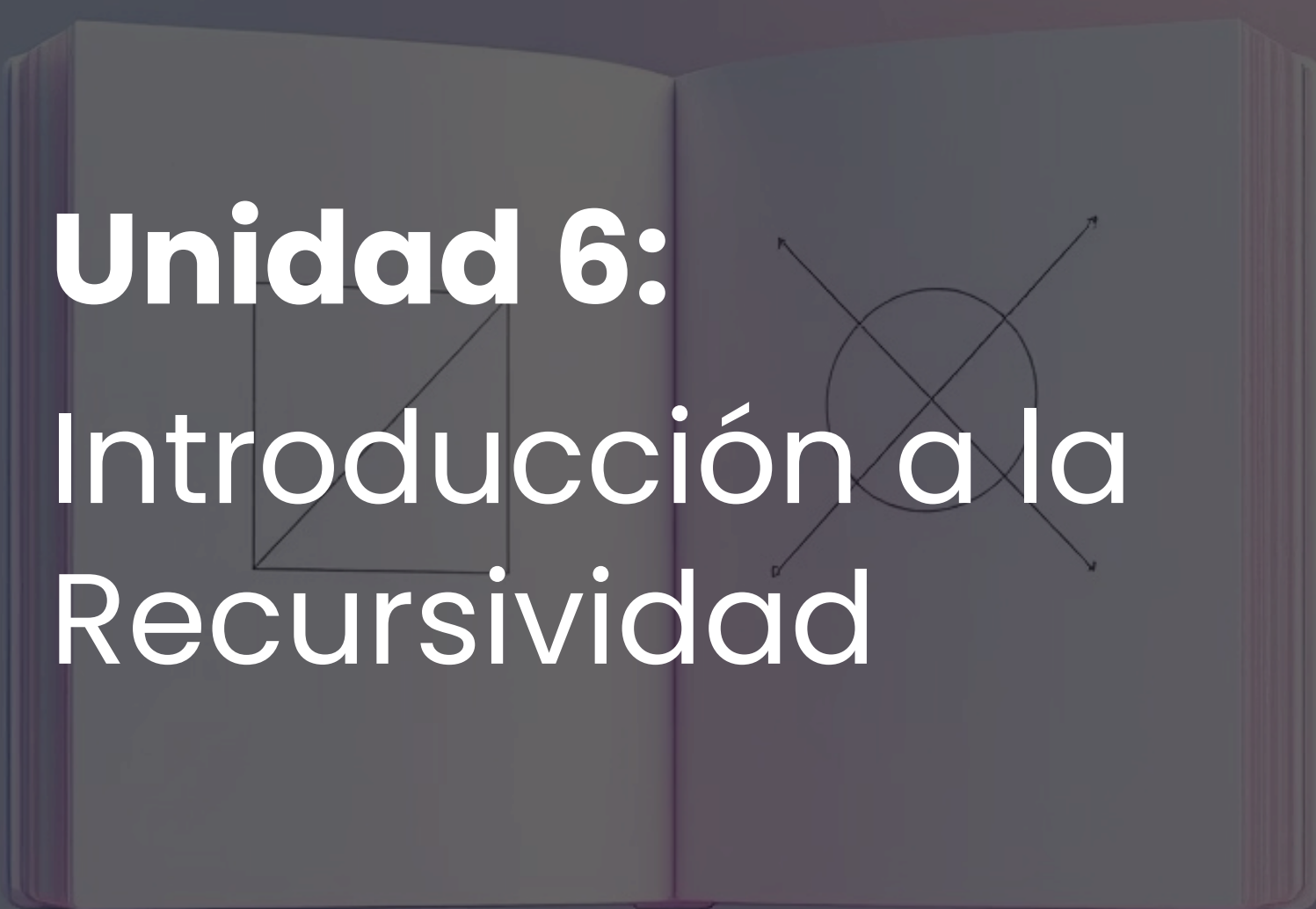


PROMETEO



Unidad 6: Introducción a la Recursividad

Módulo profesional optativo I

Técnico Superior de DAM / DAW



Sesión 23 – Concepto de recursividad y pila de llamadas

La recursividad es una de las ideas más potentes y elegantes de la programación. Consiste en que una función se llame a sí misma para resolver un problema que comparte la misma estructura que el original, pero a una escala más pequeña. Aunque la definición puede sonar abstracta, la lógica es intuitiva: divides un problema grande en problemas idénticos pero más sencillos, los resuelves y luego combinas sus resultados.

En lenguajes como Java, la recursividad forma parte esencial del pensamiento algorítmico y te permite abordar desafíos que serían muy complejos con estructuras iterativas tradicionales. Su utilidad radica en que muchos problemas del mundo real tienen una naturaleza jerárquica o fractal, como recorrer carpetas dentro de carpetas, analizar enlaces entre páginas web o procesar árboles binarios. Cuando el problema se define en términos de sí mismo, la recursión se vuelve la herramienta más natural e intuitiva.

Caso base

Indica cuándo detener la recursión

Llamada recursiva

Reduce el problema y aproxima la función al caso base

Sin caso base, la función no tiene un punto de parada y genera un proceso infinito que colapsa la pila de llamadas, provocando el conocido error `StackOverflowError`. Esta excepción no aparece por un problema de lógica matemática, sino por un límite físico del sistema: la recursividad utiliza espacio de memoria en la pila, y cada llamada consume un nuevo marco (stack frame). Si las llamadas no cesan, la memoria asignada a la pila se agota.

- ❏ El ejemplo clásico para introducir este concepto es el cálculo del factorial de un número:

```
int factorial(int n) {  
    if (n == 0) return 1; // caso base  
    else return n * factorial(n - 1); // llamada recursiva  
}
```

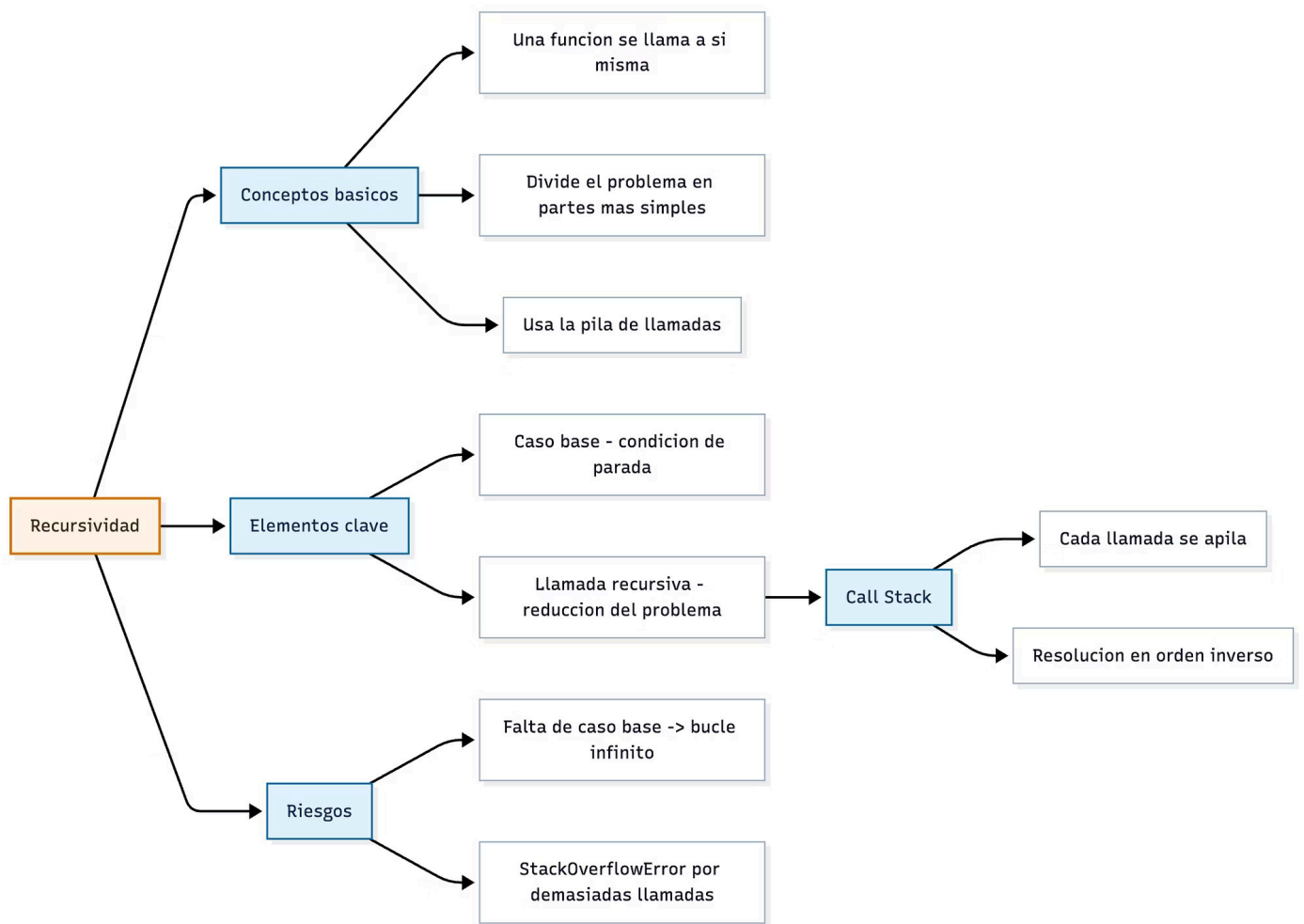
Aquí puedes observar cómo la reducción del problema ocurre sobre $n - 1$. Si la función recibe 3, la secuencia será: $\text{factorial}(3) \rightarrow \text{factorial}(2) \rightarrow \text{factorial}(1) \rightarrow \text{factorial}(0)$, y solo cuando se alcanza $\text{factorial}(0)$ comienzan las devoluciones.

Un aspecto crucial que debes interiorizar es cómo trabaja la pila de llamadas (call stack). Cada vez que una función se invoca, Java reserva un espacio en la pila para almacenar sus variables locales y el punto al que debe volver una vez termine la ejecución. En recursividad, esto ocurre muchas veces seguidas, por lo que la pila crece en forma de torre temporal. Cuando se llega al caso base, las funciones comienzan a resolverse en orden inverso al de su llamada, respetando el comportamiento LIFO (Last In, First Out).

Comprender la recursión implica entender la pila. No solo para escribir soluciones correctas, sino para depurar, anticipar errores y diseñar algoritmos más eficientes. En problemas profundos o mal definidos, la recursión puede volverse costosa, pero bien utilizada es una herramienta clara, concisa y muy poderosa en programación.



Esquema Visual



A continuación se presenta un diagrama conceptual en Mermaid que resume la estructura de la recursividad y su relación con la pila de llamadas. Puedes recrearlo directamente utilizando esta especificación.

Qué representa cada elemento

- **Recursividad:** nodo raíz que agrupa todo el concepto.
- **Conceptos básicos:** las tres ideas fundamentales: auto-llamada, división del problema y dependencia de la pila.
- **Elementos clave:** los dos componentes que toda función recursiva debe tener.
- **Riesgos:** fallos habituales al programar recursivamente.
- **Call Stack:** muestra cómo se gestionan las llamadas y por qué la recursión fluye desde el caso base hacia arriba.

Este esquema refleja la importancia de una arquitectura clara: definir bien el caso base, garantizar la reducción del problema y entender que la pila actúa como estructura de soporte.



Caso de Estudio – Google Search y la exploración recursiva de grafos

Para comprender la recursividad en un entorno real, resulta muy ilustrativo analizar cómo trabaja el motor de búsqueda de Google cuando "rastrea" la web. Google emplea algoritmos de recorridos de grafos, especialmente el conocido Depth-First Search (DFS), que es recursivo por naturaleza.

Imagina la web como un grafo gigantesco donde cada página es un nodo y cada enlace es una arista. Para indexar Internet, los robots de Google deben visitar una página, seguir sus enlaces, visitar las páginas enlazadas, y repetir el proceso tantas veces como sea necesario.

Contexto

- Google necesita analizar miles de millones de páginas.
- Cada página contiene enlaces a otras páginas, generando una estructura de grafo.
- Para decidir qué páginas indexar, en qué orden y cómo clasificarlas, el robot debe navegar por esta estructura.

Estrategia

El algoritmo DFS funciona así:

1. Visita una página.
2. Marca esa página como visitada.
3. Para cada enlace dentro de ella, llama recursivamente al mismo algoritmo, "visitando" el siguiente nodo.

Resultado

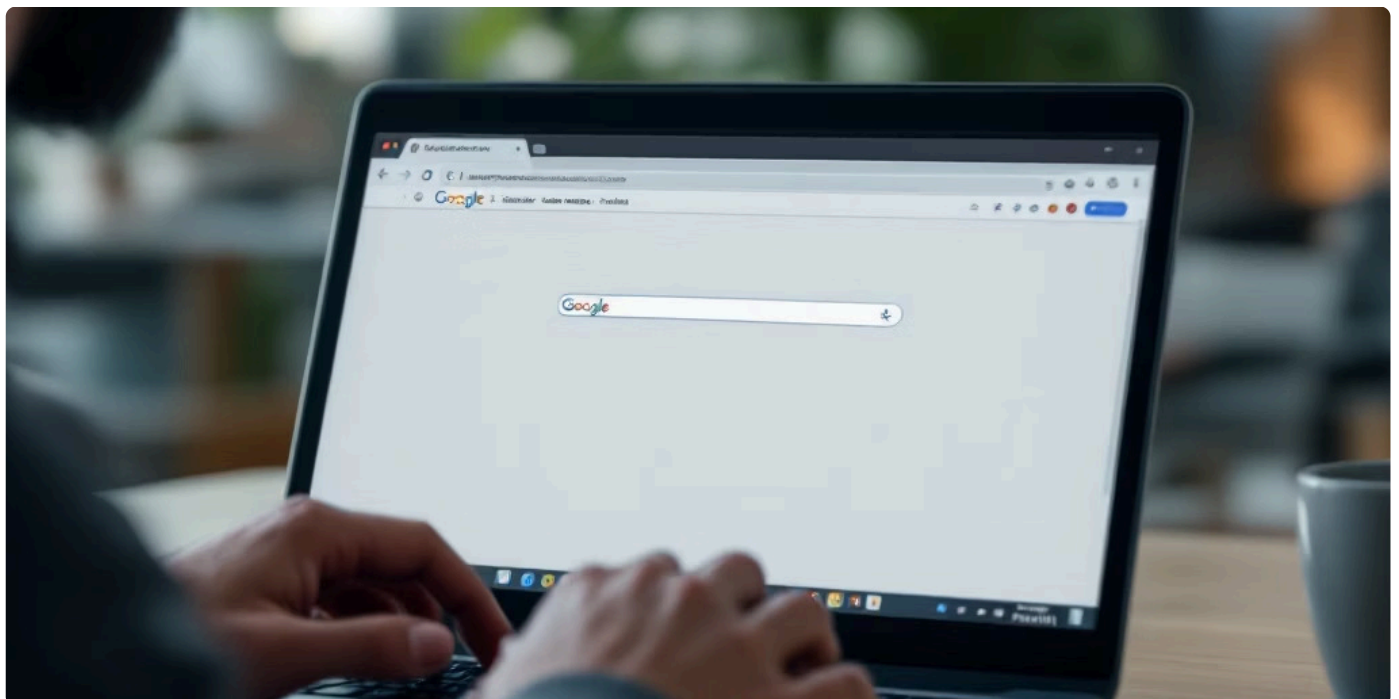
- El algoritmo explora profundamente cada rama de enlaces antes de retroceder.
- La pila de llamadas garantiza que el sistema recuerde desde qué página debe seguir trabajando.
- Aunque Google complementa DFS con algoritmos propios más sofisticados, el patrón recursivo sigue siendo una pieza fundamental en la exploración de grafos.

📄 En pseudocódigo, sería algo como:

```
void dfs(Page p) {  
    marcarComoVisitada(p);  
    for (Page enlace : p.enlaces()) {  
        if (!visitada(enlace)) {  
            dfs(enlace); // llamada recursiva  
        }  
    }  
}
```

Cada vez que dfs se invoca, el estado de la página actual se apila en la pila de llamadas. Cuando se termina con todos los enlaces de una página, la función vuelve al punto exacto donde se quedó en la llamada anterior.

Este caso muestra cómo la recursión no es solo teoría académica: está detrás de tecnologías globales que usamos cada día.



Herramientas y Consejos

A la hora de trabajar con recursividad en Java, estos consejos te ayudarán a programar de forma más clara y a evitar errores comunes.

Consejos prácticos

1 Define siempre primero el caso base.

Antes incluso de pensar en la llamada recursiva, escribe la condición que detiene la recursión. Esto reduce errores e impide recursiones infinitas.

2 Asegúrate de que cada llamada reduce el problema.

Si cada llamada no se acerca al caso base, la recursión nunca terminará. Pregúntate: ¿Esta llamada lleva al problema a un estado más sencillo?

3 Evita la recursión cuando el problema es lineal.

No uses recursividad por estética. Si el problema es claramente iterativo, un bucle será más eficiente.

4 Usa trazas para verificar el orden de ejecución.

Imprimir mensajes antes y después de la llamada recursiva te permite "ver" el flujo real del programa.

5 Entiende el límite de pila.

La recursión profunda (miles de llamadas) puede provocar un `StackOverflowError`. Es normal, y no significa que el código esté mal, sino que debes plantear alternativas.

Herramientas útiles

♦ Java Visualizer

Ideal para ver cómo se apilan las llamadas y cómo retornan.

https://www.cs.princeton.edu/~cos126/java_visualizer

♦ PythonTutor con modo Java

Simula la ejecución paso a paso y muestra la pila visualmente.

<https://pythontutor.com/java.html>

♦ IntelliJ IDEA Debugger (Step Into)

Permite seguir la ejecución línea a línea, especialmente útil para observar llamadas encadenadas.

Mitos y Realidades

A continuación se presentan los dos mitos marcados en el microlearning, con su explicación ampliada.

✗ Mito: "La recursividad siempre es más lenta."

→ FALSO. La recursión no es inherentemente más lenta. En muchos casos, un algoritmo recursivo expresa mejor la estructura del problema y evita errores. La lentitud aparece cuando el algoritmo recalcula muchas veces los mismos valores (como en Fibonacci simple), pero puede optimizarse con técnicas como memorización o pasar a una versión iterativa. La recursión es tan eficiente como su formulación lo permita, y en operaciones sobre árboles o grafos es incluso más natural que la iteración.

✗ Mito opcional: "Recursividad = elegancia pero sin riesgo."

→ FALSO. La recursión requiere comprender bien el comportamiento de la pila de llamadas. Un error en el caso base o en la reducción del problema puede provocar desbordamientos de pila. No es una técnica "inofensiva": exige precisión y análisis.

Resumen Final para el Examen

- La recursividad es cuando una función se llama a sí misma para resolver versiones más simples del problema.
- Toda función recursiva necesita caso base y llamada recursiva.
- La ejecución se sostiene sobre la pila de llamadas, que almacena cada invocación.
- Sin caso base o sin reducción del problema aparece el `StackOverflowError`.
- Es ideal para problemas jerárquicos y auto-similares.



Sesión 24 – Problemas clásicos: factorial, Fibonacci, torres de Hanói

Los problemas clásicos de recursividad, como factorial, Fibonacci y las torres de Hanói, se han convertido en referencias obligadas en el aprendizaje de programación porque condensan tres elementos esenciales del pensamiento recursivo: notar que un problema se puede definir en términos de sí mismo, asegurarse de que existe un caso base bien definido y comprender cómo se encadena la reducción del problema hasta alcanzar ese caso base.

Estos problemas funcionan como "laboratorios de recursión". Te permiten observar con claridad cómo una llamada recursiva genera otra, cómo se apilan los marcos en la call stack y cómo la solución final emerge de la resolución inversa de esas llamadas. Además, representan tres patrones muy diferentes: recursión simple (factorial), recursión doble (Fibonacci) y recursión anidada y estructural (torres de Hanói).

Entenderlos te prepara para resolver retos más complejos presentes en la informática real, como algoritmos de ordenación, recorridos de árboles, búsqueda en grafos o sistemas que procesan jerarquías.



Factorial: ejemplo de recursión directa

El factorial ($n!$) expresa naturalmente su solución en función de sí mismo: $n! = n \times (n-1)!$. Aquí la recursión aparece casi "escrita" en la definición matemática. El caso base es $0! = 1$.



Fibonacci: ejemplo de recursión doble

La serie de Fibonacci define cada valor como la suma de los dos anteriores: $F(n) = F(n-1) + F(n-2)$. El caso base se activa cuando $n \leq 1$. Este problema te ayuda a entender cómo una sola llamada genera dos llamadas más, creando un árbol de ejecución exponencial. También muestra la importancia de optimizar, porque muchos valores se recalculan repetidamente.



Torres de Hanói: recursión estructural

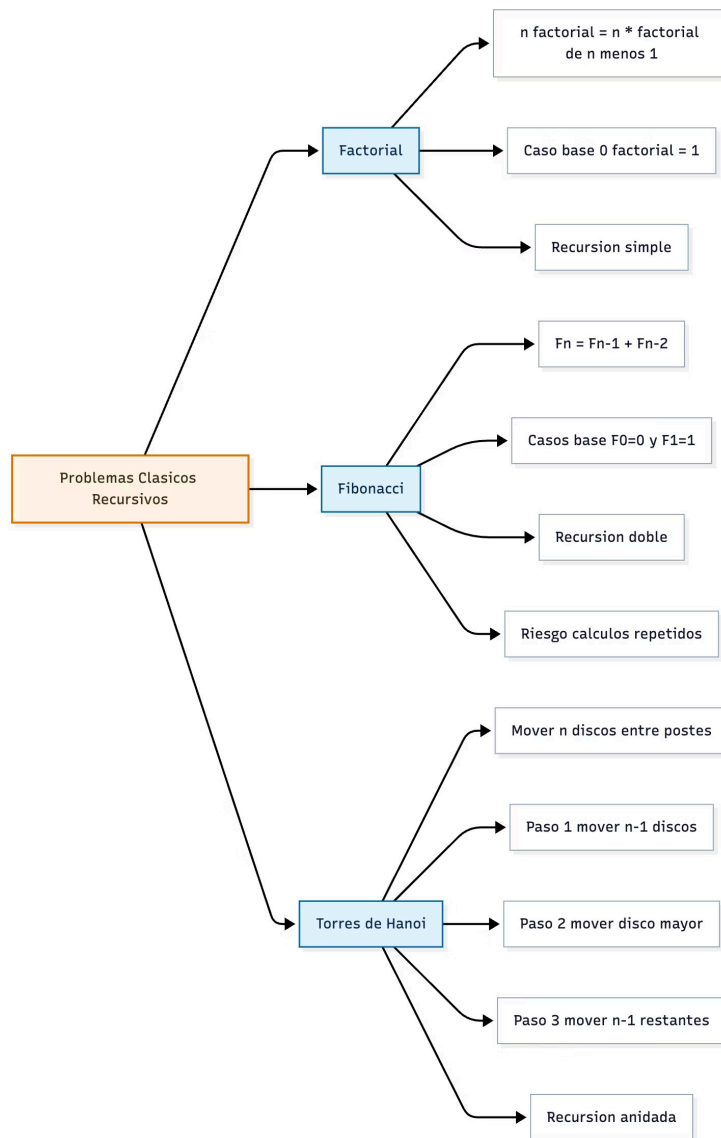
Aquí el problema no se define por una operación matemática simple sino por reglas de movimiento entre postes. Para mover n discos del poste A al C, necesitas mover $n-1$ del A al B, mover el disco mayor directamente y luego mover los $n-1$ del B al C. La recursión surge de la estructura misma del reto.

Estos tres problemas están presentes en cursos de programación desde hace décadas porque permiten ver con claridad lo que ocurre dentro de la recursión, pero también porque los principios que enseñan se aplican en áreas reales como sincronización de archivos, análisis de datos, sistemas de archivos, compresión y ordenación. En definitiva, estos ejercicios no son triviales ni meros ejemplos académicos: te dan una comprensión profunda sobre cómo funciona la recursión, cómo se gestiona la pila de llamadas y cómo elegir la estrategia más adecuada para cada tipo de problema.



Esquema Visual

A continuación tienes un diagrama en Mermaid, diseñado para ser totalmente replicable, que muestra de forma clara los tres problemas clásicos y el tipo de recursión que representan. Puedes copiarlo directamente en cualquier visor compatible con MermaidJS.



Explicación del esquema

- **Factorial:** muestra la reducción directa y lineal del problema.
- **Fibonacci:** refleja cómo una llamada genera dos, convirtiéndose en un árbol recursivo.
- **Torres de Hanói:** evidencia la necesidad de dividir el problema en tres pasos interdependientes.
- **Comparación:** cada uno representa un patrón de recursión diferente, lo que te permite entrenar la comprensión conceptual del proceso.



Caso de Estudio – Dropbox y la sincronización recursiva de carpetas

En el contexto real, uno de los ejemplos más representativos del uso de recursividad es el proceso que aplicaciones como Dropbox emplean para sincronizar archivos y subcarpetas entre dispositivos. La estructura de carpetas de un usuario no es lineal: puede contener subniveles, jerarquías profundas, carpetas anidadas y archivos de diferentes tipos. Para sincronizar estos contenidos, Dropbox debe recorrer el árbol completo.

Contexto

- Cada carpeta puede contener otras carpetas.
- La profundidad no está limitada: un usuario puede tener estructuras muy complejas.
- La sincronización requiere analizar cada elemento siguiendo la jerarquía natural del sistema de archivos.

Estrategia

Dropbox aplica un patrón recursivo:

1. Sincroniza la carpeta actual.
2. Recorre cada elemento dentro de ella.
3. Si encuentra una subcarpeta, llama recursivamente a la función de sincronización.

Resultado

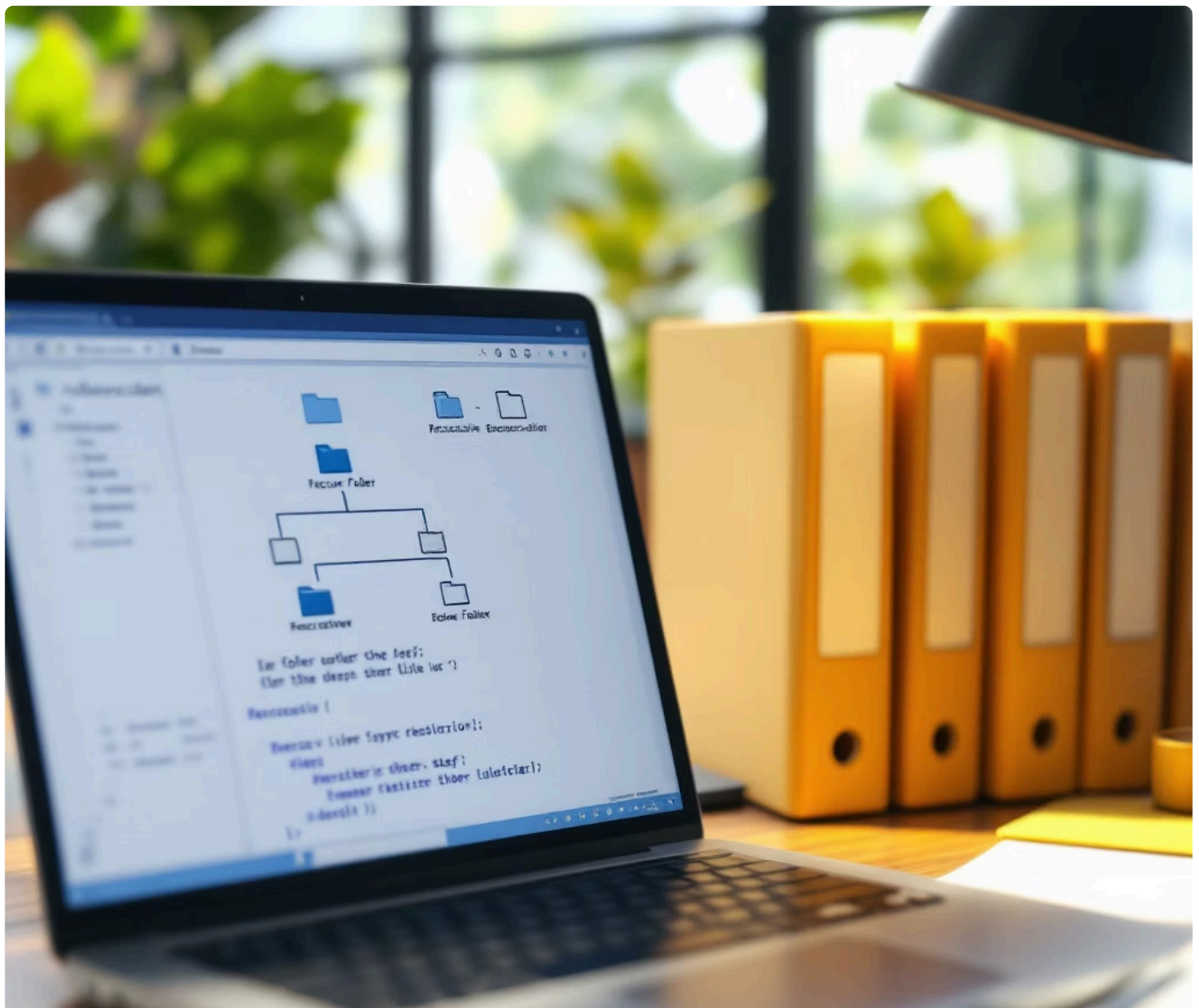
- Dropbox logra recorrer estructuras de archivos profundas sin necesidad de complejas estructuras iterativas.
- La lógica es clara, legible y extensible: nuevos niveles no requieren cambios en el algoritmo.
- El sistema escala fácilmente, ya que la recursión se adapta naturalmente a la jerarquía del directorio.

❏ Un pseudocódigo representativo:

```
void sincronizar(Carpeta c) {  
    sincronizarArchivos(c);  
    for (Carpeta sub : c.subcarpetas()) {  
        sincronizar(sub); // llamada recursiva  
    }  
}
```

Este proceso es idéntico a cómo se resuelven las torres de Hanói: la función se aplica sobre sí misma, pero enfocada a una parte más pequeña del problema.

Este caso demuestra cómo un patrón recursivo puede utilizarse hoy en día en aplicaciones masivas y distribuidas, dándole una relevancia práctica más allá de los ejercicios académicos.



Herramientas y Consejos

Para que puedas dominar estos problemas clásicos y aplicarlos con soltura en Java, aquí tienes una serie de recomendaciones prácticas y herramientas que te ayudarán a comprender mejor el comportamiento interno de cada llamada recursiva.

✓ Consejos prácticos

1 Analiza la reducción del problema antes de programar.

Pregúntate:

- ¿Cómo puedo reescribir este problema en términos del mismo problema pero más pequeño?
- ¿Esa reducción me lleva hacia un caso base real?

2 Usa trazas para visualizar el árbol de llamadas.

Insertar líneas como:

```
System.out.println("Llamando a fib(" + n + ")");
```

te permite observar en qué orden se producen las llamadas y cuándo regresan.

3 Aplica memorización en Fibonacci.

La versión básica de Fibonacci recalcula los mismos valores miles de veces. Guardar resultados previos en un array o mapa convierte un problema $O(2^n)$ en $O(n)$.

4 Dibuja el árbol de recursión de Hanói.

Visualizar el orden de los movimientos ayuda a comprender cómo se alternan los pasos.

Herramientas útiles

◆ Recursion Tree Visualizer

Ideal para mostrar gráficamente los árboles de llamadas de Fibonacci o Hanói.

◆ IntelliJ Debug – Step Into

Permite seguir cada llamada recursiva en tiempo real.

◆ Online Java Simulator

Simula la ejecución con visualización de pila y permite entender el orden de resolución.

Mitos y Realidades

Aquí se amplían los mitos señalados en el microlearning, siguiendo el formato obligatorio.

✗ Mito: "Fibonacci recursivo siempre es eficiente."

→ FALSO. La versión básica recalcula los mismos valores una y otra vez. Por ejemplo, $F(40)$ puede suponer más de mil millones de llamadas. La recursión solo se vuelve eficiente cuando se añade memorización o se convierte a versión iterativa.

✗ Mito: "Las torres de Hanói pueden resolverse sin recursión fácilmente."

→ FALSO. Aunque existe una versión iterativa, la lógica natural del problema es recursiva. La recursión permite expresar los tres pasos esenciales (mover $n-1$ discos, mover el mayor, mover $n-1$ discos) de forma clara y directa.

📄 Resumen Final para el Examen

- Factorial: recursión simple → $n * \text{factorial}(n-1)$ con caso base $0!$.
- Fibonacci: recursión doble → cálculo basado en dos llamadas; necesita optimización.
- Hanói: recursión estructural → mover n discos requiere tres pasos recursivos.
- Todos requieren caso base, reducción del problema y control del número de llamadas.
- Problemas clásicos → base sólida para entender estructuras reales como carpetas, grafos y árboles.





Sesión 25 – Recursividad vs iteración: cuándo usar cada una

Cuando empiezas a programar, una de las primeras decisiones que te encuentras al resolver un problema repetitivo es elegir entre dos enfoques: iteración y recursividad. Ambos son técnicas que permiten repetir pasos, pero cada una lo hace de forma diferente y con implicaciones muy distintas en memoria, legibilidad y desempeño.

La iteración

Se basa en bucles como `for`, `while` o `do-while`. Estos bucles se ejecutan mientras una condición sea verdadera, modificando variables de control para avanzar hacia la finalización. Es una estructura muy eficiente en memoria, porque el programa reutiliza el mismo bloque de código sin generar nuevas llamadas ni ocupar espacio adicional en la pila. Por eso, los lenguajes imperativos como Java, C o Python la consideran la opción más económica en la mayoría de casos.

La recursividad

Funciona cuando una función se llama a sí misma hasta alcanzar un caso base que detiene la cadena de llamadas. Cada vez que la función se invoca, Java crea un nuevo "marco" en la pila de llamadas, donde almacena datos locales y el punto de retorno. Esta pila crece a medida que crece la recursión, lo que le permite recordar el estado de cada ejecución anterior.

Esta característica es precisamente lo que hace que la recursión sea tan natural para problemas donde el propio problema está compuesto por versiones más simples de sí mismo: árboles, carpetas, grafos, estructuras jerárquicas o algoritmos como QuickSort.

Ambas técnicas son equivalentes desde el punto de vista de expresividad: lo que se puede resolver con recursividad se puede resolver con iteración y viceversa. Sin embargo, no son intercambiables sin coste. La recursividad es más elegante, más cercana a la definición matemática de muchos problemas y más fácil de leer cuando trabajas con estructuras anidadas. Pero a la vez requiere un control cuidadoso del caso base y del consumo de memoria.

- ❏ Por ejemplo, calcular el factorial de un número mediante iteración es directo:

```
int factorialIterativo(int n) {  
    int res = 1;  
    for (int i = 1; i <= n; i++)  
        res *= i;  
    return res;  
}
```

Pero la versión recursiva expresa mejor el concepto matemático, aunque consume más pila:

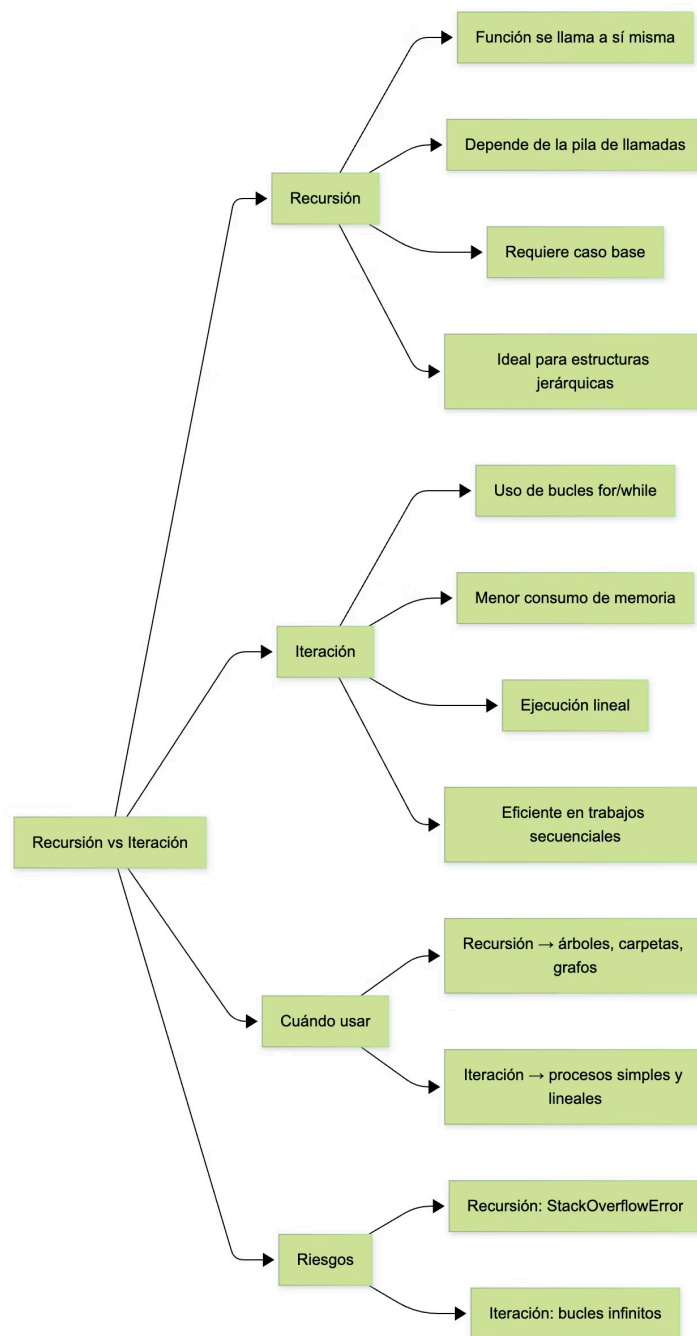
```
int factorial(int n) {  
    if (n == 0) return 1;  
    return n * factorial(n - 1);  
}
```

En problemas lineales —como sumar elementos de un array, recorrer una lista o leer líneas de un archivo— normalmente la iteración es más eficiente. En cambio, cuando te enfrentas a estructuras donde cada elemento puede tener subelementos, como un sistema de archivos, un árbol genealógico o una estructura XML, la recursión se vuelve el modelo mental más natural.

Usar una u otra no depende solo del rendimiento, sino de cómo quieres expresar el problema y del riesgo de errores. La recursión, mal diseñada, puede causar el temido `StackOverflowError`. La iteración, por otro lado, puede acabar siendo mucho más aparatosa y difícil de leer cuando el problema tiene una estructura jerárquica. Dominar la diferencia entre ambos enfoques te permite escribir código más claro, más eficiente y adaptado al contexto de cada problema.

Esquema Visual

Aquí tienes un esquema detallado en formato Mermaid, diseñando una representación precisa de los elementos clave de recursión e iteración y cuándo usar cada técnica.



Qué representa cada parte del esquema

- **Recursión:** muestra los requisitos fundamentales (caso base y pila) y su ámbito ideal.
- **Iteración:** refleja eficiencia y linealidad, lo que la convierte en la opción óptima para procesos simples.
- **Cuándo usar:** establece criterios precisos para tomar la decisión correcta.
- **Riesgos:** advierte de los problemas más comunes al aplicar cada técnica.

Este diagrama sintetiza la diferencia conceptual y práctica entre ambos enfoques.



Caso de Estudio – GitHub y el recorrido de directorios

Para entender cómo se usan recursión e iteración en aplicaciones reales, examinemos un caso concreto: GitHub. Cuando GitHub permite navegar por un repositorio, visualizar carpetas o listar archivos, internamente trabaja con dos enfoques según el tipo de operación.

01

✓ Recursividad para recorrer directorios

Los repositorios pueden contener carpetas dentro de carpetas sin un límite claro. Cada carpeta puede contener:

- código fuente,
- tests,
- documentación,
- módulos internos,
- otros directorios.

Esta estructura es naturalmente jerárquica, lo que se presta a una solución recursiva.

02

✓ Iteración para operaciones lineales

En cambio, GitHub utiliza iteración en operaciones secuenciales, como:

- leer un archivo línea a línea,
- procesar commits ordenados temporalmente,
- recorrer un listado plano de issues o pull requests.

03

✓ Resultado

GitHub combina ambos enfoques según la naturaleza del problema:

- Recursión cuando la estructura es profunda y jerárquica.
- Iteración cuando la estructura es lineal, secuencial y predecible.

❏ El pseudocódigo del recorrido típico es:

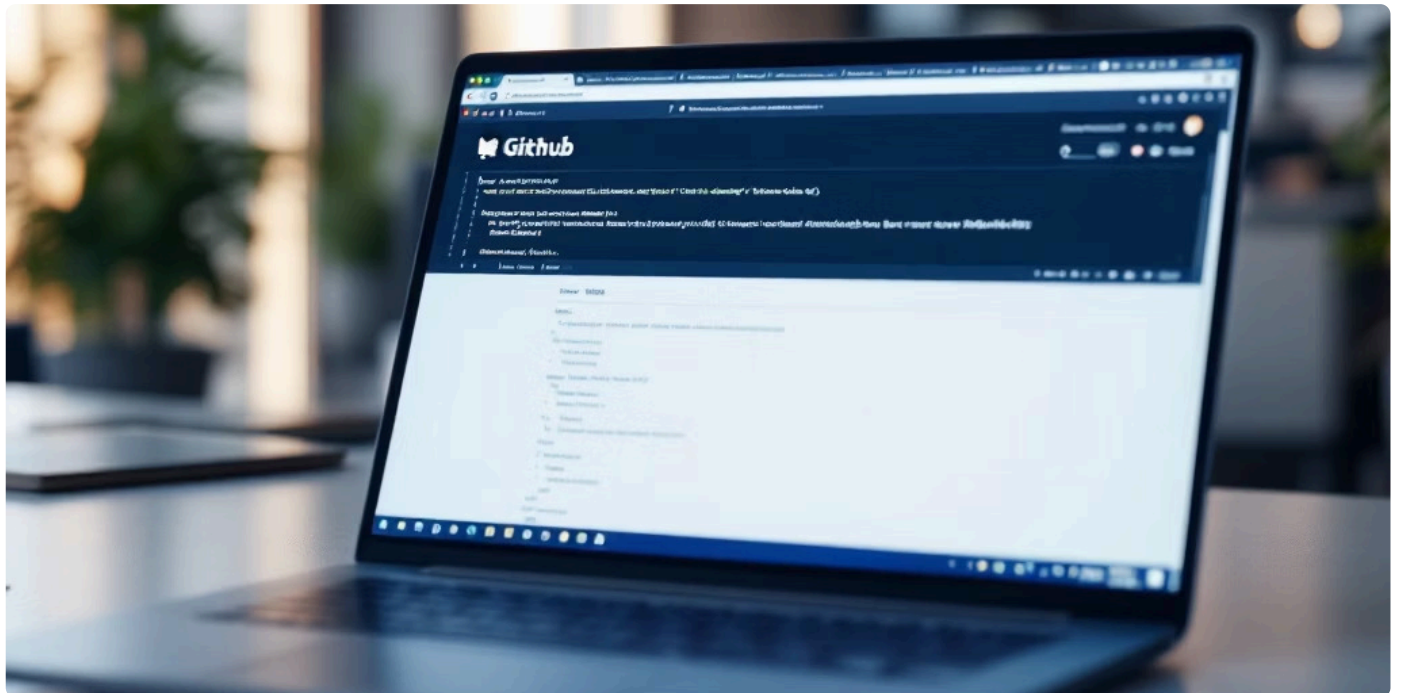
```
void recorrerDirectorio(Directorio d) {  
    listarArchivos(d);  
    for (Directorio sub : d.subdirectorios()) {  
        recorrerDirectorio(sub); // llamada recursiva  
    }  
}
```

Ejemplo de iteración:

```
while (linea != null) {  
    procesar(linea);  
    linea = lector.nextLine();  
}
```

Por cada subcarpeta encontrada, GitHub vuelve a llamar a la función de recorrido. Esto simplifica el código, lo hace más expresivo y adapta la solución al modelo jerárquico del repositorio.

Este caso ilustra exactamente cuándo cada técnica es más útil y por qué ninguna es "mejor" que la otra por sí sola.



Herramientas y Consejos

Consejos prácticos

1 Usa recursión para estructuras jerárquicas.

Si el problema tiene subniveles o nodos hijos, como árboles o carpetas, la recursión es la herramienta más natural.

2 Usa iteración para procesos lineales o repetitivos.

Búsquedas simples, acumuladores, contadores, recorridos de arrays... todo lo que siga un patrón lineal suele beneficiarse de la iteración.

3 Considera el límite de la pila.

Si el problema puede generar miles de llamadas, usa iteración o una pila manual para evitar desbordamientos.

4 Conviértelo a iteración cuando sea necesario.

Puedes simular una recursión con una pila manual:

```
Stack pila = new Stack<>();
pila.push(raiz);

while (!pila.isEmpty()) {
    Nodo actual = pila.pop();
    // procesar
    for (Nodo hijo : actual.hijos())
        pila.push(hijo);
}
```

Esto evita el `StackOverflowError` y te da más control sobre memoria y rendimiento.

Herramientas útiles

◆ Visual Recursion Simulators

Simuladores online que muestran cómo crece y se vacía la pila durante una llamada recursiva.

◆ JProfiler

Excelente para medir consumo de memoria y observar el tamaño de la pila en tiempo real.

◆ Debug Step Into (IntelliJ/Eclipse)

Permite entrar dentro de cada llamada y ver cómo se encadenan las ejecuciones.

Mitos y Realidades

Aquí tienes exactamente dos mitos, desarrollados según el formato obligatorio:

✗ Mito: "La recursión siempre reemplaza a los bucles."

→ FALSO. Aunque todo bucle se puede expresar de forma recursiva, la recursión no siempre es la mejor opción. Los bucles consumen menos memoria y suelen ser más eficientes para procesos lineales. La recursión destaca únicamente cuando el problema tiene una estructura jerárquica o auto-similar. No existe una solución universal: la clave es elegir la estrategia adecuada al contexto.

✗ Mito: "La iteración es más segura que la recursión."

→ FALSO. La iteración también puede fallar: los bucles infinitos son igual de peligrosos que un `StackOverflowError`. Ambos enfoques requieren control adecuado. La seguridad depende del diseño del algoritmo, no del enfoque en sí.

Resumen Final para el Examen

- Recursión → una función que se llama a sí misma; ideal para estructuras jerárquicas.
- Iteración → uso de bucles; más eficiente en memoria y mejor para problemas lineales.
- Recursión usa pila de llamadas y depende del caso base.
- Iteración evita `StackOverflow` y es más rápida en procesos sencillos.
- Ambos enfoques pueden resolver los mismos problemas, pero no con el mismo coste.